

P2468R2

The Equality Operator You Are Looking For

Published Proposal, 2022-01-13

This version:

<http://wg21.link/p2468r2>

Authors:

[Barry Revzin](#)

[Bjarne Stroustrup](#)

[Cameron DaCamara](#)

[Daveed Vandevoorde](#)

[Gabriel Dos Reis](#)

[Herb Sutter](#)

[Jason Merrill](#)

[Jonathan Caves](#)

[Richard Smith](#)

[Ville Voutilainen](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper details some changes to make rewriting equality in expressions less of a breaking change

Table of Contents

- 1 **Rewritten equality/inequality in expressions**
 - 1.1 Compiler disagreements

- 1.2 Existing code impact
 - 1.2.1 Open source sampling
- 1.3 Proposed resolution
 - 1.3.1 First resolution
 - 1.3.2 Second resolution

2 Programming Model

3 Wording

4 Appendix

- 4.1 Code patterns which fail to compile even under new rules
- 4.2 Code patterns which fail at runtime
- 4.3 Code patterns which fail due to `<=>`

5 Revision History

- 5.1 Revision 1
- 5.2 Revision 2

§ 1. Rewritten equality/inequality in expressions

§ 1.1. Compiler disagreements

After the merge of P1185R2, C++20 changed the meaning of `==` and `!=` expressions and how overload resolution applies to them. The following was valid C++17 code:

```
struct S {  
    bool operator==(const S&) { return true; } // The non-const member is  
    bool operator!=(const S&) { return false; } // The non-const member is  
};  
bool b = S{} != S{};
```

GCC accepts this (despite overload resolution being ambiguous under C++20).

Clang accepts with a warning indicating the rewrite resulting in ambiguous overload resolution.

```
<source>:5:14: warning: ISO C++20 considers use of overloaded operator '!='  
    bool b = S{} != S{};
```

MSVC accepts this by eagerly dropping rewritten candidates.

The reason this sample should result in an ambiguity and does not get to the intended tiebreaker step [\[over.match.general\]/2.8](#) is because ambiguity is actually the result of finding best conversions. During the best conversions step we discover that the candidate `operator!=(const S&)` and rewritten candidate `operator==(const S&)` are equally as good as each other. To the compiler, it is as if the user had written:

```
struct S {  
    friend bool f(S, const S&) { ... }  
    friend bool f(const S&, S) { ... }  
};  
bool b = f(S{}, S{});
```

Which is a clear ambiguity during the comparison of best conversions.

§ 1.2. Existing code impact

It is clear to the author(s) of this paper, since the compiler vendors feel the need to bypass the overload resolution rules defined in the standard that there must be some valid code patterns for which the new overload resolution rules unintentionally break.

Using MSVC we implemented the strict rules defined by [\[over.match.general\]](#) and ran the compiler against a series of open source projects to gather data on how impactful the strict application of the new overload resolution rules can be.

§ 1.2.1. Open source sampling

The results of applying the strict rules are as follows:

Total Projects	Failed Projects
-----------------------	------------------------

Many of the failures are caused by the first code sample mentioned. Other failures include:

```
template <typename T>
struct Base {
    bool operator==(const T&) const;
    bool operator!=(const T&) const;
};

struct Derived : Base<Derived> { };

bool b = Derived{} == Derived{};
```

In this case the user intended to use some common base class to implement all of the comparison operators. Because the new operator rewriting rules will also add the synthesized candidate to the overload set the result becomes ambiguous when trying to compare best conversions. Both the regular candidate and the synthesized candidate contain a derived-to-base conversion which makes one not strictly better than the other.

We feel that the code above should be **accepted**.

```
template <bool>
struct GenericIterator {
    using ConstIterator = GenericIterator<true>;
    using NonConstIterator = GenericIterator<false>;
    GenericIterator() = default;
    GenericIterator(const NonConstIterator&);

    bool operator==(ConstIterator) const;
    bool operator!=(ConstIterator) const;
};
using Iterator = GenericIterator<false>;

bool b = Iterator{} == Iterator{};
```

This is a scenario where the user is depending on implicit conversions to get the desired effect of comparisons being compared as a `ConstIterator`. The issue is that the converting constructor

enables conversions on both sides which are no better than the other when considering the reversed candidate.

We feel that the code above should be **accepted**.

```
struct Iterator {
    Iterator();
    Iterator(int*);
    bool operator==(const Iterator&) const;
    operator int*() const;
};

bool b = nullptr != Iterator{};
```

This code was relying on the fact that an implicit conversion gives the user `!=` for free. The issue is that this implicit conversion creates a scenario where the user-defined conversion operator creates a case where it is not a better choice than the reversed parameter candidate rewrite where a temporary `Iterator` type is constructed from `nullptr`.

We feel that this is a case where C++20 helps the user identify possible semantic problems. It is a case we should **reject** despite being observed in two different projects.

```
using ubool = unsigned char;

struct S {
    operator bool() const;
};

ubool operator==(S, S);

ubool b = S{} != S{};
```

Based on the merge of [P1630R1](#) there is a case added for the rewritten candidate chosen must return cv-`bool` and if it does not the candidate is rejected, but because that condition happens after overload resolution the code author ends up seeing:

```
error C2088: '!=': illegal for struct
```

While this paper does not want to tackle that wording, we feel that the code above should remain **rejected** until a future update.

§ 1.3. Proposed resolution

§ 1.3.1. First resolution

To help address the issues mentioned above we gathered input from the following individuals regarding how GCC and Clang implements overload resolution:

- Jason Merrill
- Richard Smith

The implementation approach taken by GCC appeared to be the most permissive and applied seemingly reasonable rules which allow most of the above samples to compile. The GCC approach is generalized as:

Before comparing candidates for best conversion sequences, compare each candidate to each other candidate (or function template they are specializations of) and if the parameter types match and one is a rewritten candidate the rewritten candidate is not considered for later tiebreakers.

After implementing the rule above in MSVC we obtained the following results from running that compiler against open source projects:

Total Projects	Failed Projects
59	10 (<i>down from 20</i>)

It is immediately clear that the GCC implementation was on the correct path to a good solution. The GCC approach ticked all the boxes for code we wanted to compile (mentioned above) while maintaining the spirit of the original operator rewriting proposal [P0515R3](#) by allowing heterogeneous comparisons.

§ 1.3.2. Second resolution

Upon more reflection over the first resolution we further refined that rule into something more targeted at operator rewrites for `==` as opposed to the more general rule as implemented by MSVC for the second resolution.

First, the rule, as suggested by Richard Smith:

When considering adding a rewritten operator to the candidate set, if the rewrite target is `operator==`, and a matching `operator!=` is declared (that is: if you take the declaration of the `operator==` and replace the name with `operator!=`, would that declaration redeclare anything?), do not add the rewritten candidate to the candidate set. (With no other changes to the C++20 rules.)

After implementing the above suggestion in MSVC we have gathered the following data after running the compiler through a number of open source projects (this number is different from the tables above due to more projects added since the original run):

Total Projects	Failed Projects
110	8

Most of the breakage due to the rule above were around the following pattern:

```
struct S {  
    bool operator==(const S&);  
};
```

Where the type does not define a corresponding `operator!=` to disable the rewrite behavior.

Based on the results and the principled approach of the latter rule we propose this resolution for standardization.

§ 2. Programming Model

With proposal suggested in "Second resolution" the programming model for C++20 becomes:

- If you want an `operator==` that is used for rewrites (automatically reversed, and `!=` automatically generated), write only an `operator==`, and make sure its return type is `bool`.
- If you want an `operator==` that is not used for rewrites, write both an `operator==` and a matching `operator!=`.
- `operator<=>` is always used for rewrites (from `<`, `<=`, `>`, `>=`); if you don't want rewrites, don't write an `operator<=>`.

For migration:

- To keep the behavior the same, make sure every `operator==` has a matching `operator!=`.

- Once you're happy for rewrites to start being performed, remove the `operator!=`.

Surprises and caveats:

- If you have an `operator==` with no matching `operator!=`, *and* you migrate from C++17 to C++20, *and* your `operator==` is not symmetric (either intentionally or accidentally -- usually due to a missing `const`), *and* you use an `==` or `!=` in a way where the rewritten form of that operator is a better match, then you can have ambiguity errors or a silent change in behavior when migrating.
- If you use the `operator==` before the compiler sees the matching `operator!=` declaration (eg, from within the definition of the same `operator==`, for a non-member non-template `operator==`), you can still get rewrites.

§ 3. Wording

Note: we intend this wording to be merged as a DR against C++20.

Change in 12.2.2.3 [over.match.oper] paragraph 3:

The rewritten candidate set is determined as follows:

- For the relational ([expr.rel]) operators, the rewritten candidates include all non-rewritten candidates for the expression $x \leq y$.
- For the relational ([expr.rel]) and three-way comparison ([expr.spaceship]) operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each non-rewritten candidate for the expression $y \leq x$.
- For the != operator ([expr.eq]), the rewritten candidates include all non-rewritten candidates for the expression $x == y$ that are rewrite targets with first operand x (see below).
- For the equality operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each non-rewritten candidate for the expression $y == x$ that is a rewrite target with first operand y.
- For all other operators, the rewritten candidate set is empty.

A non-template function or function template F named operator== is a rewrite target with first operand o unless a search for the name operator!= in the scope S from the instantiation context of the operator expression finds a function or function template that would correspond ([basic.scope.scope]) to F if its name were operator==, where S is the scope of the class type of o if F is a class member, and the namespace scope of which F is a member otherwise. A function template specialization named operator== is a rewrite target if its function template is a rewrite target.

[Example:

```
struct A {};  
    template<typename T> bool operator==(A, T); // #1  
    bool a1 = 0 == A(); // OK, calls reversed #1  
    template<typename T> bool operator!=(A, T);  
    bool a2 = 0 == A(); // error, #1 is not a rewrite target  
  
    struct B {  
        bool operator==(const B&); // #2  
    };  
    struct C : B {  
        C();  
        C(B);  
        bool operator!=(const B&); // #3  
    };  
    bool c1 = B() == C(); // OK, calls #2; reversed #2 is not a candidate  
    bool c2 = C() == B(); // error, ambiguous between #2 found when searching
```

```

struct D {};
template <typename T>
bool operator==(D, T); // #4
inline namespace N {
    template <typename T>
    bool operator!=(D, T); // #5
}
bool d1 = 0 == D(); // OK, calls reversed #4; #5 does not forbid #4 as a

```

-- end example]

[Note 2: A candidate synthesized from a member candidate has its implicit object parameter as the second parameter, thus implicit conversions are considered for the first, but not for the second, parameter. — end note]

Change in 10.4.3.3 [glob.module.frag] paragraph 3:

- S contains an expression E of the form postfix-expression (expression-listopt) whose postfix-expression denotes a dependent name, or for an operator expression whose operator denotes a dependent name, and D is found by any name lookup performed for the corresponding name in an expression synthesized from E by replacing each type-dependent argument or operand with a value of a placeholder type with no associated namespaces or entities, or [Note: This includes the lookup for operator== performed when considering rewriting an != expression, the lookup for operator<=> performed when considering rewriting a relational comparison, and the lookup for operator!= when considering whether an operator== is a rewrite target.]

§ 4. Appendix

§ 4.1. Code patterns which fail to compile even under new rules

```

template<class Derived>
struct Base {
    int operator==(const double&) const;

```

```

    friend inline int operator==(const double&, const Derived&);
};

struct X : Base<X> { };

bool b = X{} == 0.;

```

In C++17 and before, the member `operator==` was selected. In C++20, the friend `operator==`, with reversed parameter order, is selected because it does not require a derived-to-base conversion. Rewriting to this `operator==` is not disabled because no corresponding `operator!=` is declared. This rewrite fails due to requirements of rewritten operator returning cv-`bool`.

```

struct Base {
    bool operator==(const Base&) const;
    bool operator!=(const Base&) const;
};

struct Derived : Base {
    Derived(const Base&);
    bool operator==(const Derived& rhs) const {
        return static_cast<const Base&>(*this) == rhs;
    }
};

```

The code above fails due to relying on a derived-to-base conversion on both sides of the comparison (once the synthesized candidate is taken into account). If one imagines a similar scenario:

```

bool b1 = Derived{} == Base{};
bool b2 = Base{} == Derived{};

```

This would also be ambiguous even without the definition of `Derived::operator==` and in our view is considered a bugfix in C++20 which forces the user to carefully consider how objects are being compared.

§ 4.2. Code patterns which fail at runtime

In C++20, because of how candidate sets are expanded there are a few scenarios where a new candidate introduced is then selected as the best candidate without any code change. In many cases this can be fine but there are a few cases we identified as being potentially problematic for code authors.

```
struct iterator;
    struct const_iterator {
        const_iterator(const iterator&);
        bool operator==(const const_iterator &ci) const;
    };

    struct iterator {
        bool operator==(const const_iterator &ci) const { return ci == *this;
    };
```

In C++17 the sample above would compile and the function selected for the comparison `ci == *this` would be `const_iterator::operator==(const const_iterator&)`. In C++20 the function chosen for the same comparison is the (rewritten) function `iterator::operator==(const const_iterator&)` using the reversed parameter order rule. The chosen function in C++20 causes a new runtime failure where the comparison will cause infinite recursion where there wasn't one before.

Another example:

```
struct U { };

    struct S {
        template <typename T>
        friend bool operator==(const S&, const T&) { return true; }

        friend bool operator==(const U& u, const S& s) {
            return s == u;
        }
    };

    bool b = U{} == S{};
```

In C++17 the user intended for comparisons to be dispatched to the templated `operator==`. In C++20 the templated `operator==` is considered to be a worse match based on [\[over.match.best.general\]/2.4](#) which is a tiebreaker before rewritten candidate tiebreakers, which makes `operator==(const U& u, const S& s)` the best match for the comparison `s == u`.

§ 4.3. Code patterns which fail due to `<=>`

The paper also acknowledges other types of breakages involving the spaceship operator such as the reported issue in [reddit](#). While this paper makes no attempt to address the concerns in the thread there is the potential for fixing such issues through diagnostics provided by the compiler or tooling.

§ 5. Revision History

§ 5.1. Revision 1

- Refine "second resolution" / updated wording to reflect the new rule.

§ 5.2. Revision 2

- Wording changes as suggested in CWG at https://wiki.edg.com/bin/view/Wg21telecons2022/Teleconference2022-01-13?twiki_redirect_cache=5b60b829fb121fadf6474aaa8dceaf05
- Add reference to P1185R2 as the root cause
- Explicitly mention this is a DR to C++20